

1. PRESENTACIÓN DEL ESPACIO CURRICULAR

Espacio curricular: Paradigmas de Programación				
Código SIU-guaraní:	00913	Horas Presenciales	90	Ciclo lectivo: 2026
Carrera:	Lic. en Ciencias de la Computación	Plan de Estudio:	Res 004/23 -CD	
Dirección a la que pertenece	Elija un elemento.	Bloque/ Trayecto	Algoritmos y lenguajes	
Ubicación curricular:	5to Semestre	Créditos: 9	Formato Curricular	Teoría/práctica
EQUIPO DOCENTE	Nombre:	Correo:		
Cargo: Profesor Titular	Nombre: Pablo Javier Vidal	Correo: pablo.vidal@ingenieria.uncuyo.edu.ar		
Cargo: Jefe de Trabajos Practico	Nombre: Laura Noussan Lettry	Correo: lauranlweb@gmail.com		

Fundamentación

Paradigmas de la Programación busca abordar distintos modelos de programación o paradigmas para resolver problemas y evaluar la forma en que se resuelven los problemas. Uno de los principales objetivos de esta asignatura es que el estudiante sea capaz de elegir el modelo adecuado para resolver cada problema. Los paradigmas de programación son pilares que definen el desarrollo informático de los programas y por extensión el desarrollo computacional. La aparición de cada uno de ellos ha marcado un hito en el modelo de desarrollo de un programa informático. En esta asignatura se estudiarán los paradigmas de programación más representativos de las diferentes formas de modelar un programa informático.

Es objetivo entonces Paradigmas de Programación proporcionar una introducción a diversos aspectos fundamentales del diseño de soluciones en las Ciencias de la Computación para estudiantes de grado que sea lo suficientemente completa como para abordar sus implicaciones en los avances tecnológicos que se han suscitado y se suscitaran.

Aportes al perfil de egreso (De la Matriz de Tributación)

CE - Competencias de Egreso Específicas	CE-GT Competencias Genéricas Tecnológicas	CE-GSPA Competencias Sociales – Político - Actitudinales
CE 1.1. Especificar, proyectar y desarrollar sistemas de información. CE 1.3. Especificar, proyectar y desarrollar software.	CE-GT 1 Identificar, formular y resolver problemas de informática. CE-GT 4 Utilizar de manera efectiva las técnicas y herramientas de aplicación en la informática.	CE-GSPA 7 Comunicarse con efectividad. CE-GSPA 9 Aprender en forma continua y autónoma.

--	--	--

Expectativas de logro (del Plan de Estudios)

- Comprender conceptos fundamentales de los diferentes paradigmas y lenguajes de programación identificando aspectos comunes, y aplicarlos en situaciones problemas de interés.
- Analizar y aplicar criterios que permitan identificar el paradigma adecuado según el tipo de problema a resolver.

Contenidos mínimos (del Plan de Estudios)

Conceptos de paradigmas de programación. Relaciones entre los distintos paradigmas y lenguajes.

Correlativas (Saberes previos/ posteriores del Plan de Correlatividades)

Saberes previos: Algoritmos y estructura de datos II, Arquitectura de computadoras I - Sistemas operativos - Programación II - Lógica

Saberes posteriores: Ingeniería de Software II. Programación Paralela y Distribuida. Compiladores

2. RESULTADOS DE APRENDIZAJE

RA1. Analizar un lenguaje de programación desde el punto de vista de su paradigmas para relacionarlo con las conceptos teóricos que permiten generar soluciones a partir de la relación lenguaje/maquina.

RA2 Identificar las principales características y limitaciones de los distintos lenguajes y paradigmas de programación

RA3 Reconocer el proceso de trabajo de cada lenguaje y entender su funcionamiento mediante el uso del sistema operativo.

3. CONTENIDOS/SABERES (Organizados por unidades, ejes u otros)

Unidad 1: Introducción

El proceso de desarrollo de software. Dominios de aplicación. Lenguajes de programación. Evolución de los lenguajes de programación. Criterios de evaluación. Introducción a Paradigmas de programación.

Unidad 2: Paradigma Funcional

Calculo lambda. Transparencia Referencial. Programas como funciones. Evaluación perezosa. Funciones de orden superior. Introducción al lenguaje Haskell.

Unidad 3: Paradigmas de Programación

Paradigmas de Programación. Comparación. Abstracción, polimorfismo de los lenguajes de programación.

Unidad 4: Principios de diseño de los lenguajes

Contexto de un lenguaje: Variables, Tipado, Ligaduras, Alcance. Diseño y estructura de los lenguajes de programación. Modelos de traducción: Compilador e Intérprete. Tabla de Símbolos. Lexica, Sintaxis y semántica.

Unidad 5: Subprogramas, ambientes y parametrización

Pasaje de datos, de subprogramas y de tipos. Pasaje de parámetros por referencia, por copia, por nombre y parámetros anónimos. Punteros. Parámetros de subprogramas pasados como parámetros. Lenguaje Ensamblador. Registro de Activación de tamaño fijo y variable. Semántica de los subprogramas. Mecanismos de paso de parámetros. Estructura de bloques. Jerarquía de bloques, implícita y jerarquía de bloques explícita. Ambientes de Ejecución. Variables locales y globales. Valores iniciales. Sintaxis del pasaje de parámetros. Semántica del pasaje de parámetros.

Unidad 6: Acceso a Memoria e interacción a Sistema Operativo. (SYSCALLS)

Memoria libre en lenguajes dinámicos. CIR. Recolección de basura. Identificación de los fragmentos de memoria libre no referenciados. Pila para la recolección de basura. Contadores de referencias. Sistemas operativos con particiones fijas. Hebras. Scheduling. Coordinación entre la administración de memoria del Sistema Operativo y la del Lenguaje.

4. MEDIACION PEDAGOGICA (metodologías, estrategias, recomendaciones para el estudio)

Por tratarse de una asignatura con importante respaldo conceptual, y para facilitar la apropiación de los conocimientos, se aplicará la ejemplificación y demostración de manera metódica. Los contenidos del programa se presentan y analizan en las clases teóricas y se consolidan con los trabajos prácticos. Existe una estrecha relación entre la teoría y práctica, a su vez la práctica fundamenta sus procedimientos en la teoría.

Los ejercicios están orientados al análisis o entendimiento de las soluciones propuestas y a la concepción o diseño para responder a nuevos requerimientos y problemas. Para ello el material de práctica es seleccionado de manera de cubrir un amplio espectro temático y muestra diferentes niveles de dificultad. Algunos ejercicios son resueltos completamente en clase, y otros son encaminados para ser completados por los alumnos fuera de clase.

Estrategias metodológicas y de enseñanza

Las seis horas semanales que tiene asignada la materia se distribuyen de la siguiente manera:

- Clases teóricas de 3 horas para presentación y exposición de los temas, con el uso de pizarrón con apoyo de proyección de imágenes y computadora. Dichas clases incluyen gran contenido de forma práctica y serán participativas y de debate. Se desarrollarán de manera expositiva los conceptos fundamentales, con presentación de casos de estudio prácticos de aplicación inmediata del tema presentado. Se promoverá la investigación y la búsqueda de información por parte del alumno. Se exigirá un trabajo continuo al alumno tanto dentro del aula como fuera de ella.
- Clases prácticas de tres horas de ejercitación para resolución de problemas y consultas trabajando en el aula. Estas clases incluirán presentación del tema, casos de estudio abiertos a debate y de construcción colectiva, y consulta individual de ejercicios. Se incluyen actividades de laboratorio para aplicar diferentes lenguajes de programación (Python, Java, Prolog, Haskell, lenguaje ensamblador, C, entre otros) como herramienta instrumental de resolución de problemas de acuerdo al paradigma que se esté estudiando.

5. INTENSIDAD DE LA FORMACION PRACTICA

Ámbito de formación práctica	Carga horaria	
	Presencial	No presencial
Formación Experimental		

Resolución de problemas de la vida real en informática	45	
Actividades de proyecto y diseño		
Práctica profesional Supervisada		
Otras actividades		
Total	45	

6. SISTEMA DE EVALUACIÓN

6.1. Criterios de evaluación

En la evaluación se busca reconocer la capacidad del alumno para aplicar los contenidos teóricos en situaciones prácticas. Para ello se desarrollan distintas evaluaciones escritas y orales teniendo a alcanzar los resultados de aprendizaje requeridos.

6.2. Condiciones de regularidad

Para regularizar la asignatura el alumno deberá aprobar dos (2) exámenes parciales escritos. Los alumnos que desaprobén (nota inferior a 6) alguno de los exámenes tendrá acceso a un recuperatorio final integrador.

En base a los resultados de las evaluaciones el alumno quedará como alumno regular, cuando haya aprobado los correspondientes exámenes parciales, o su recuperatorio global. No hay régimen especial para alumnos recursantes.

6.3. Condiciones de promoción

No existe régimen de promoción para esta asignatura.

6.4. Régimen de acreditación para

- **Promoción directa**

No existe promoción directa

- **Alumnos regulares**

El examen final en el caso de alumnos regulares es de tipo integrador teórico-práctico, de forma oral y escrita, sobre cualquiera de los temas desarrollados en la asignatura. El examen final se rinde a programa completo, exigiéndose el nivel superior correspondiente. Independientemente que se hayan tomado o no en las evaluaciones parciales. Todos los temas evaluados deben conocerse en al menos un 60% del alcance desarrollado en la materia. Podrán rendir examen final aquellos alumnos regulares.

- **Alumnos libres**

El examen final para alumnos libres es de tipo integrador teórico-práctico, de forma escrita, sobre cualquiera de los temas desarrollados en la asignatura. En particular, se desarrollarán ejercicios prácticos que demuestren alcanzar un nivel de conocimiento completo de la asignatura. En caso de

que los ejercicios estén aprobados con una nota superior o igual a 6 (SEIS) el alumno libre deberá rendir un examen teórico-práctico oral en el mismo momento y lugar según los criterios de evaluación mencionados previamente. El examen final libre se rinde a programa completo, exigiéndose el nivel superior correspondiente.

- A.** Estudiante libre en el espacio curricular por no haber cursado la asignatura.
- B.** Estudiante libre en el espacio curricular por insuficiencia; *es decir, haber cursado la asignatura, y haber aprobado actividades específicas del espacio curricular y no haber cumplido con el resto de las condiciones para alcanzar la regularidad.*
- C.** Estudiante libre en el espacio curricular por pérdida de regularidad (LPPR) por vencimiento de la vigencia de la misma y no haber acreditado la asignatura en el plazo estipulado.
- D.** Estudiante libre en el espacio curricular por pérdida de regularidad (LPPR), por haber rendido CUATRO (4) veces la asignatura, en condición de estudiante regular, sin lograr su aprobación.

7. BIBLIOGRAFIA

Titulo	Autor /es	Editorial	Año de Edición	Ejemplares Disponibles	Sitios digitales
Programming Languages Pragmatics	Michael L. Scott	Morgan Kaufmann	2015	1	-
Concepts of Programming Languages	Sebesta Robert W.	Addison Wesley	2015	2	-
Practical Foundations for Programming Languages	Harper Robert	University Press	2013	1	

7.1. Recursos digitales del espacio curricular (enlace a aula virtual y otros)

Esta asignatura tiene su espacio en el Moodle de aula abierta de la Facultad de Ingeniería.

<https://aulaabierta.ingenieria.uncuyo.edu.ar/course/view.php?id=1652>

8. FIRMAS

VºBº DIRECTOR/A DE CARRERA

Fecha



DOCENTE RESPONSABLE A CARGO

Fecha 13/10/2023